

Enterprise Knowledge Assistant — Your Roadmap

A step-by-step path from where you are now to a finished, demo-able project. Follow it top to bottom — each phase depends on the one before it.

Where you are right now: ✅ Phase 1 complete (auth working: register, login, JWT, protected routes)

Phase 1 — Foundation & Auth ✅ DONE

- FastAPI project structure
- User model + database
- Register / Login / JWT / `get_current_user`
- Fixed bcrypt/passlib version conflict

You're already past this. Nothing to do here except keep `requirements.txt` updated as you add packages.

Phase 2 — Basic LLM Q&A (Week 1 goal — do this next)

Why first: before you can search your own documents, you need to confirm you can talk to the LLM at all and that your auth protects the endpoint.

Steps:

1. Get an OpenAI (or Azure OpenAI) API key.
2. Add `OPENAI_API_KEY` to `.env` and `config.py`.
3. Install `openai` SDK.
4. Build a `/chat/ask` endpoint that takes a question and returns an LLM answer.
5. Protect it with your existing `get_current_user` dependency.
6. Test in Swagger: login → Authorize with token → ask a question → get an answer.

Done when: you can log in and get a real AI-generated answer back through your API.

(I already gave you the full code for this step above — go implement it before moving on.)

Phase 3 — Document Ingestion (Week 2, part 1)

Why: the assistant is useless without your company's documents to answer from.

Steps:

1. Create an `/upload` endpoint that accepts a PDF file.
2. Save the file (locally in a folder for now, e.g. `storage/uploads/`).

3. Extract text from the PDF (library: `pypdf` or `PyMuPDF`).
4. Split the text into chunks (~500-1000 tokens each, with slight overlap) — this is called **chunking**. Library: `langchain-text-splitters` or write a simple splitter yourself.
5. Store the chunks in your database temporarily (a `DocumentChunk` table: id, document_id, text, page_number).

Done when: you upload a PDF and can see its text broken into chunks stored in your DB.

Phase 4 — Embeddings & Vector Search (Week 2, part 2)

Why: this is what lets the assistant find the *relevant* chunk instead of reading the whole document every time — this is the "R" in RAG (Retrieval-Augmented Generation).

Steps:

1. For each chunk, generate an embedding (a vector of numbers representing meaning) using OpenAI's embeddings API (`text-embedding-3-small`) or Azure OpenAI equivalent.
2. Store these vectors in a vector database. For a beginner project, start simple:
 - **Easiest:** store vectors in your Postgres DB using the `pgvector` extension.
 - **Alternative (matches playbook):** Azure AI Search or Pinecone — more setup, but production-style.
3. Build a `/chat/ask` upgrade: when a question comes in, embed the question too, then search for the top 3-5 most similar chunks.
4. Pass those chunks into the LLM prompt along with the question ("Answer using only this context: ...").

Done when: asking a question about your uploaded PDF returns an answer that's actually grounded in that document, not just general LLM knowledge.

Phase 5 — Citations (Suggested Enhancement, do early — it's high value & not hard)

Why: this is what makes the tool trustworthy and is explicitly called out as differentiating.

Steps:

1. When returning the answer, also return which chunk(s) it came from (document name + page number).
2. Return this as structured JSON: `{ "answer": "...", "sources": [ { "document": "...", "page": 4 } ] }`.

Done when: every answer comes with a "this came from Document X, page 4" reference.

Phase 6 — Tool Calling / Agent Behavior (Week 3)

Why: turns this from "a search box with AI" into an actual *agent* that can decide what to do.

Steps:

1. Pick a framework: Microsoft Agent Framework (matches the C# path) or Semantic Kernel/LangChain if staying in Python.
2. Give the agent a "tool" — e.g. a function `search_documents(query)` that runs your Phase 4 retrieval.
3. Let the LLM decide *when* to call that tool vs. answer directly (function calling / tool calling).
4. Optionally add a second tool, e.g. querying a mock internal database or API, so the agent has more than one option to choose from — this is what makes it "autonomous."

Done when: the agent decides on its own whether it needs to search documents to answer, rather than you always forcing a search.

Phase 7 — Memory & Feedback Loop (Week 4)

Steps:

1. Add conversation memory so follow-up questions ("what about last month?") understand prior context. Store recent messages per user/session (Redis is ideal, but even a DB table works for a first project).
2. Add a thumbs up/down endpoint on each answer, logged to the DB — this is your "feedback-driven re-ranking" hook for later.
3. Add basic logging/monitoring (even simple request logging is fine; Application Insights is the production-grade option).

Done when: the assistant remembers context within a session and you're capturing user feedback.

Phase 8 — Polish & Enhancements (pick 1-2, don't do all at once)

Choose based on time available — these are what turn it from "prototype" into "compelling product":

- Multi-format ingestion (Word, Excel, Slack exports, OCR for scanned images)
 - Access-control aware retrieval (users only see documents they're permitted to)
 - Usage analytics dashboard (most-asked questions, time saved)
 - Teams/Slack integration
-

Phase 9 — Deliverables (required by the playbook)

1. **Working prototype** — your deployed/running app.
 2. **Architecture diagram** — I can help you generate this once Phase 4 is done (I can build it as a diagram directly in chat).
 3. **GitHub repository** — push your code, include the `requirements.txt` and a clear `README.md` explaining setup.
 4. **Blog post** — write-up of what you built and what you learned (I can help you draft this).
 5. **Recorded demo video** — screen recording walking through register → login → upload doc → ask question → get cited answer.
-

How to work with me going forward

Come back after finishing each phase and tell me you're ready for the next one (e.g. "I finished Phase 2, chat/ask works"). I'll give you the specific code and instructions for that phase only — this keeps things beginner-friendly and avoids overwhelming you with the whole project at once.

Right now: go build Phase 2 (LLM Q&A endpoint) using the code from my previous message, then come back.